

Practice 11 详解答案

A. 单项选择题

1. **C**: 指针变量保存内存地址。
2. **B**: `&` 是取地址运算符。
3. **B**: `ptr` 保存 `score` 的地址。
4. **C**: `*` 用于解引用, 访问该地址处的对象。
5. **A**: `ptr` 的值是 `age` 的地址, `*ptr` 的值才是 `20`。
6. **C**: `string* note` 是指向 `string` 的指针。
7. **B**: 空指针没有可访问的有效目标地址。
8. **A**: 指针能通过地址间接访问对象。
9. **C**: 先检查 `ptr != nullptr`。
10. **B**: 变量存储数据值, 指针存储数据所在的地址。

B. 填空题

1. 地址
2. `&`
3. `*`
4. `nullptr`
5. 变量 (对象)

C. 判断题

1. **对**: 指针有自己的类型、名称和存储空间。
2. **对**: `&score` 取得 `score` 的地址。
3. **对**: `*ptr` 访问 `ptr` 指向对象的值。
4. **错**: 解引用空指针是未定义行为, 可能导致程序崩溃。
5. **对**: `int* p;` 声明了一个 `int` 指针, 但它尚未初始化。
6. **对**: 引用和指针都可关联已有对象。
7. **对**: 指针可以改为保存另一个变量的地址。
8. **错**: 引用初始化后始终表示同一个对象; 赋值会修改对象值, 不会重新绑定。
9. **对**: 若 `p` 指向有效的 `int`, 该语句会修改目标变量。
10. **对**: 指针体现了变量、地址与内存访问之间的关系。

D. 代码阅读

1. 输出

```
int value = 42;  
cout << value;
```

输出： 42 。

2. ptr 存储什么

```
int score = 88;  
int* ptr = &score;
```

ptr 存储 **score 的内存地址**。 ptr 是地址， *ptr 才是地址处的值 88 。

3. 输出

```
cout << *ptr;
```

输出： 88 。 *ptr 解引用后访问 score 。

4. number 的最终值

```
int number = 10;  
int* ptr = &number;  
*ptr = 25;
```

最终值是 25 。 ptr 指向 number ，所以 *ptr = 25 修改了 number 。

5. 代码问题

```
int* ptr = nullptr;  
cout << *ptr;
```

ptr 没有指向有效对象，却被解引用，属于未定义行为。必须先检查：

```
if (ptr != nullptr) {  
    cout << *ptr;  
}
```

E. 简答题

1. 变量、地址和指针的关系

变量保存一个值，例如 `int score = 100;`。该变量在内存中有一个地址，`&score` 可以取得该地址。指针是专门保存地址的变量，例如 `int* ptr = &score;`。通过 `*ptr` 可以访问或修改 `score`。

2. 引用和指针的区别

引用是对象的别名，声明后必须绑定一个对象，使用时和普通变量一样：`note += " text"`。指针保存对象地址，可以为 `nullptr`，也可以改指向其他对象；访问其目标需要解引用：`*notePtr += " text"`。调用引用函数传变量，调用指针函数传地址。

3. 为什么先检查空指针

空指针不代表任何有效对象。解引用它会让程序访问无效内存，结果不可预测。因此解引用前使用 `if (ptr != nullptr)`，只在指针有效时访问目标。

F. 编程题

1. 第一个指针

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int score = 100;  
    int* scorePtr = &score;  
  
    cout << "Score: " << score << '\n';  
    cout << "Address: " << &score << '\n';  
    cout << "Pointer Value: " << *scorePtr << '\n';  
}
```

地址的具体数值每次运行可能不同；`scorePtr` 与 `&score` 显示的是同一个地址。

2. 通过指针修改

```
#include <iostream>
using namespace std;

void increaseScore(int* scorePtr)
{
    if (scorePtr != nullptr) {
        *scorePtr += 10;
    }
}

int main()
{
    int score = 90;
    increaseScore(&score);
    cout << score << '\n';
}
```

输出：100。调用处传入 &score，函数内通过 *scorePtr 修改原变量。

3. 基于指针的笔记编辑器

```
#include <iostream>
#include <string>
using namespace std;

void addImportantTag(string* notePtr)
{
    if (notePtr != nullptr) {
        *notePtr += " [IMPORTANT]";
    }
}

int main()
{
    string note = "Finish project";
    addImportantTag(&note);
    cout << note << '\n';
}
```

输出：Finish project [IMPORTANT]。

4. 指针安全检查器

```
#include <iostream>
using namespace std;

int main()
{
    int* ptr = nullptr;

    if (ptr == nullptr) {
        cout << "Pointer is null.\n";
    } else {
        cout << "Pointer is valid.\n";
    }
}
```

该程序只检查指针，绝不执行 `*ptr`。

5. 引用与指针

```
#include <iostream>
#include <string>
using namespace std;

void updateByReference(string& note)
{
    note += " [Ref]";
}

void updateByPointer(string* note)
{
    if (note != nullptr) {
        *note += " [Ptr]";
    }
}

int main()
{
    string note = "Learning C++";
    updateByReference(note);
    updateByPointer(&note);
    cout << note << '\n';
}
```

输出： Learning C++ [Ref] [Ptr] 。

- `updateByReference` 使用引用参数 `string& note`，调用时写 `updateByReference(note)`。
- `updateByPointer` 使用指针参数 `string* note`，调用时传地址 `updateByPointer(¬e)`。
- 指针函数内需要 `*note` 访问实际字符串，并应先判断它不是 `nullptr`。

G. 设计与思考挑战

版本 **B** 和 **C** 会真正修改原始笔记：

- 版本 A 的 `string note` 是副本，只修改副本，函数返回后原笔记不变。
- 版本 B 的 `string& note` 是原笔记的别名，`+=` 直接修改原笔记。
- 版本 C 的 `string* note` 保存原笔记地址，`*note +=` 解引用后修改原笔记；调用时要传入 `&原笔记`，且应保证指针有效。

对于必定存在、需要直接修改的对象，引用写法通常更简洁；需要表达“可能没有对象”或需要改变指向时，指针更合适。